



Transaction Recovery (1)

(Chapter 19 - Recovery System)

Failure Classification

Failure Classification

- **Transaction failure :**
 - **Logical errors:** transaction cannot complete due to some internal error condition
 - **System errors:** the database system must terminate an active transaction due to an error condition (e.g., deadlock)
 - **System crash:** a power failure or other hardware or software failure causes the system to crash.
 - **Fail-stop assumption:** non-volatile storage contents are assumed to not be corrupted by system crash
 - Database systems have numerous integrity checks to prevent corruption of disk data
 - **Disk failure:** a head crash or similar disk failure destroys all or part of disk storage
 - Destruction is assumed to be detectable: disk drives use checksums to detect failures
-
- **Transaction failed** - only that particular transaction failed, not the entire system.
 - **System crash** - the entire system go down, the computer system crash, but the disk still okay.
 - **Disk failure** - mass storage crash

In this chapter, we mainly deal on transaction failure and system crash → this 2 kind of failure, the dbms will automatically recover transaction for us without using any backup.

- However, for disk failure, backup are required.



Transaction Failure

Transaction Failure Example

- There was computer center that update lot of data overnight → there are many operation in a transaction, actually, they put 40 database operation in 1 transaction. The transaction deal with million of rows.
- They also keep journal like first statement is finished, second is finish....
- In the morning, transaction failed, on the screen, it said transaction failed → 0 records update, but when they look at the journal → 90 percent of work has been done.



What happened? → 0 record update, transaction failed → the whole thing rolled back.

How to prevent this?

- We found that a log file was full (log file keep temporary activity) → log file could not accommodate all those activity.
- Problem → not enough space for log file, too many activity.
- why they put 40 operation in 1 transaction? log file could not accommodate that, transaction failed.
 - they said because it an transaction → they used old definition of transaction (A file which keeps changes to be made to the master file)
 - and there no integrity constraint that they want to violate
 - actually, each of 40 operations can be separate → set auto commit on.
 - became 40 transactions, each commit separately, and because each transaction commit separately → the log file was not full (commit → clear the log file,...)

2 types of cause

- Logical errors → from poor coding (infinite loop, read bad input, ...)
- System errors → deadlock or log file was full, etc.



System Crash

- power failure is very common, A power failure or hardware or software failure cause system to crash

- Too many user, don't have enough memory to accommodate them, some system shutdown themselves, possible

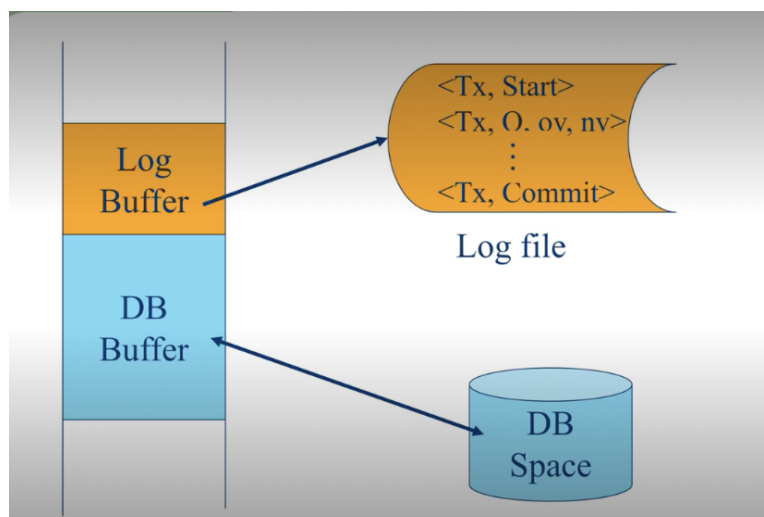
How to prevent or solve, what are the problem actually?

Log-based Recovery

- Problem Statements
- Problem 1
 - Committed Transactions
 - No Database Modification
- Problem 2
 - Uncommitted Transactions
 - Database Already Modified

- Problem 1 → how to recover those commit transaction that loss in main memory, so that they appear permanent on DB space. (commit before output)
- Problem 2 → you have uncommitted transaction, unfinished, system went down, how to cancel out those(output before commit)(database modification that need to be roll back

Most popular solution → Log-based Recovery



In principle, we have database buffer and db space.

- We have another buffer call log buffer and log file

Storage Structure

Storage Structure

- **Volatile storage:**
 - Does not survive system crashes
 - Examples: main memory, cache memory
 - **Nonvolatile storage:**
 - Survives system crashes
 - Examples: disk, tape, flash memory, non-volatile RAM
 - But may still fail, losing data
 - **Stable storage:**
 - A mythical form of storage that survives all failures
 - Approximated by maintaining multiple copies on distinct nonvolatile media
 - See book for more details on how to implement stable storage
- Stable storage
 - Redundant storage
 - Some DBMS implement stable storage for us, so it can be implemented by software possible

Data Access

- input and output are transfer between database on mass storage (disk) and the main memory

Data Access

- **Physical blocks** are those blocks residing on the disk.
- **Buffer blocks** are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
 - **input** (B) transfers the physical block B to main memory.
 - **output** (B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- We assume, for simplicity, that each data item fits in, and is stored inside, a single block.

- when the book or user manual said block → they means database block not OS block
- what's database block
 - smallest unit that transfer from mass storage to main memory.
 - smallest unit the the DBMS read from mass storage.
 - that database block may comprised of many OS block.
- input(B) - transfers the physical block B to main memory.
- output(B) - transfer the buffer block B to the disk.

Who control input, output activity?

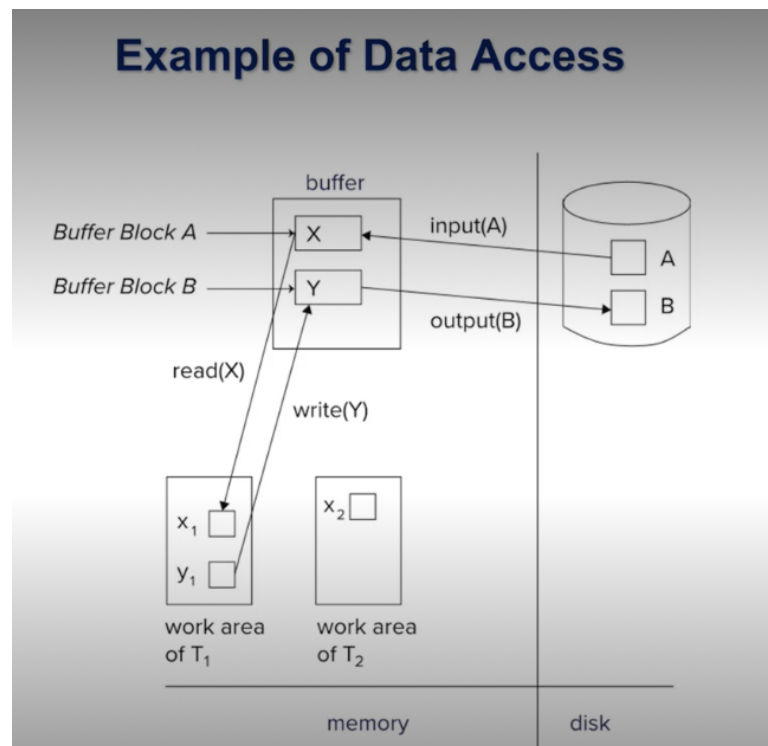
- **DBMS and the OS**, not our application program.

Data Access (Cont.)

- Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.
 - T_i 's local copy of a data item X is called x_i .
- Transferring data items between system buffer blocks and its private work-area done by:
 - **read**(X) assigns the value of data item X to the local variable x_i .
 - **write**(X) assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
 - Note: **output**(B_x) need not immediately follow **write**(X). System can perform the **output** operation when it deems fit.
- Transactions
 - Must perform **read**(X) before accessing X for the first time (subsequent reads can be from local copy)
 - **write**(X) can be executed at any time before the transaction commits

read in here too, when should it output, when no body else want to use the data or it full and have to get rid of the data first. Output are not control by application program. Input maybe initiate by read or write statement.

- The book refer to the term read and write separately from input, output.



- Database buffer is in main memory of database server (if you have), part of main memory that keep data for an entire database system.
- Transaction work area can be in the same machine or also different machine
- Read → to move data from buffer block to transaction work area
- write → to write data from transaction work area to database buffer.

Our program (sql statement) never works with a disk → it work with a buffer.

Input and output between buffer and DB Space are done by the DBMS and OS.

when your transaction want to read something, and the data already in the buffer → you read it

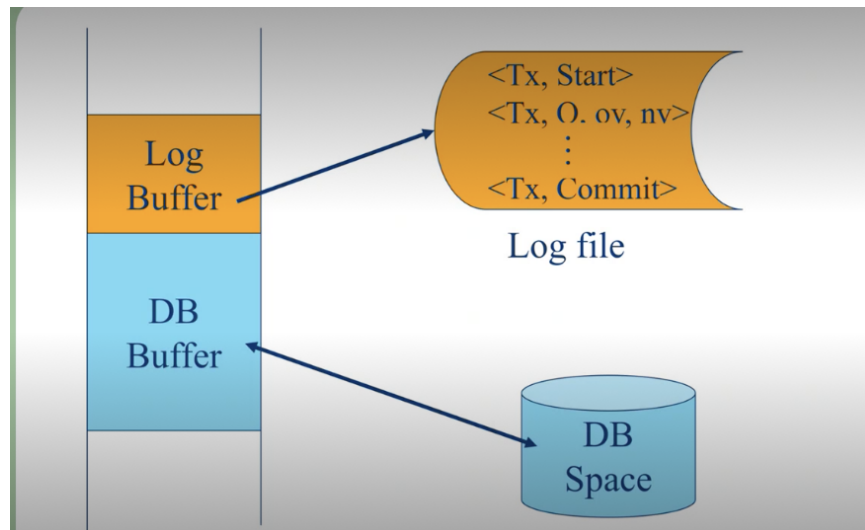
- but if it not there, system will input the data from disk to buffer (read may initiate input)

say you want to write small y to big Y, but if the big Y is not yet in the buffer → The DBMS need to inout it to the buffer

- Both read and write may initiate input activity

Log based recovery

- in here, log file is introduced
- have separate log buffer to keep log record → those log record will be output to the log file during execution of transaction



During operations, we may input data from db space to the buffer, if the data require by the application, DBMS and OS may input the data. And DBMS and OS may output the data from the buffer to DB Space.

<Tx, Start> → Transaction start (system generate transaction id)

<Tx, Q, ov, nv> → Q is a data item, ov is old value(value before operation, nv → value after operation

we keep on doing this and at the end, we said commit, there are commit record output to the log file. Log file record changes change to database.

- Tx → Transaction identifier

Example of Log record



Immediate Database Modification Example

Log	Write	Output
$\langle T_0 \text{ start} \rangle$		
$\langle T_0, A, 1000, 950 \rangle$		
$\langle T_0, B, 2000, 2050 \rangle$		
	$A = 950$ $B = 2050$	
$\langle T_0 \text{ commit} \rangle$		
$\langle T_1 \text{ start} \rangle$		
$\langle T_1, C, 700, 600 \rangle$		
	$C = 600$	
$\langle T_1 \text{ commit} \rangle$		B_B, B_C B_A
■ Note: B_X denotes block containing X.		

Annotations:
 - B_C output before T_1 commits
 - B_A output after T_0 commits

Database System Concepts - 7th Edition 19.15 ©Silberschatz, Korth and Sudarshan



Immediate DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

Recovery actions in each case above are:

- (a) undo (T_0): B is restored to 2000 and A to 1000, and log records $\langle T_0, B, 2000 \rangle$, $\langle T_0, A, 1000 \rangle$, $\langle T_0, \text{abort} \rangle$ are written out
- (b) redo (T_0) and undo (T_1): A and B are set to 950 and 2050 and C is restored to 700. Log records $\langle T_1, C, 700 \rangle$, $\langle T_1, \text{abort} \rangle$ are written out.
- (c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then C is set to 600

- The log record may not come in sequence like this, it doesn't have to be like T_0 then $T_1 \rightarrow$ it all mix up, depend on event
- commit record in log file $\rightarrow$ physical commit

$\langle T_x, Q, OV, NV \rangle$
 $\vdots$
 $\langle T_x, \text{commit} \rangle$ ————— physical commit
 * A tx commits physically after its $\langle T_x, \text{commit} \rangle$ is successfully output to the log file.

- if commit already output to the log file → it mean transaction already commit physically.
- log file == stable storage
- stable storage → assume not to be loss.

Log based Recovery

Log-Based Recovery

- A **log** is a sequence of **log records**. The records keep information about update activities on the database.
 - The **log** is kept on stable storage
- When transaction T_i starts, it registers itself by writing a $\langle T_i, \text{start} \rangle$ log record
- Before T_i executes **write**(X), a log record $\langle T_i, X, V_1, V_2 \rangle$ is written, where V_1 is the value of X before the write (the **old value**), and V_2 is the value to be written to X (the **new value**).
- When T_i finishes its last statement, the log record $\langle T_i, \text{commit} \rangle$ is written.
- Two approaches using logs
 - Immediate database modification
 - Deferred database modification.



Transaction Commit

- A transaction is said to have committed when its commit log record is output to stable storage
 - All previous log records of the transaction must have been output already
- Writes performed by a transaction may still be in the buffer when the transaction commits, and may be output later

- output can be done after commit
- output can be perform after or before commit

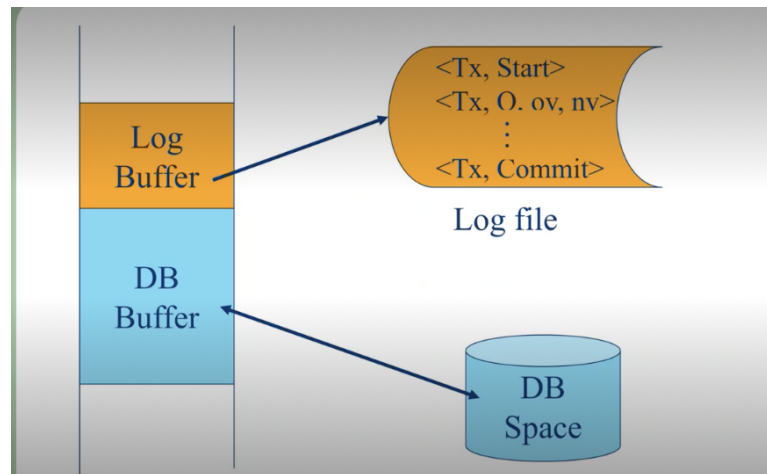
Log	Database
<T ₀ start>	
<T ₀ , A, 1000, 950>	
<T ₀ , B, 2000, 2050>	
	A = 950 B = 2050
<T ₀ commit>	
<T ₁ start>	
<T ₁ , C, 700, 600>	
	C = 600
<T ₁ commit>	

- This figure is a little bit misleading
 - it look like database is modify after the log
 - log → modify → commit (it look like we have to modify database first before commit) but in fact, log record must go to log file first, DB buffer go to db space
 - it show look like we have to output before commit → but actually, output can be done after commit.

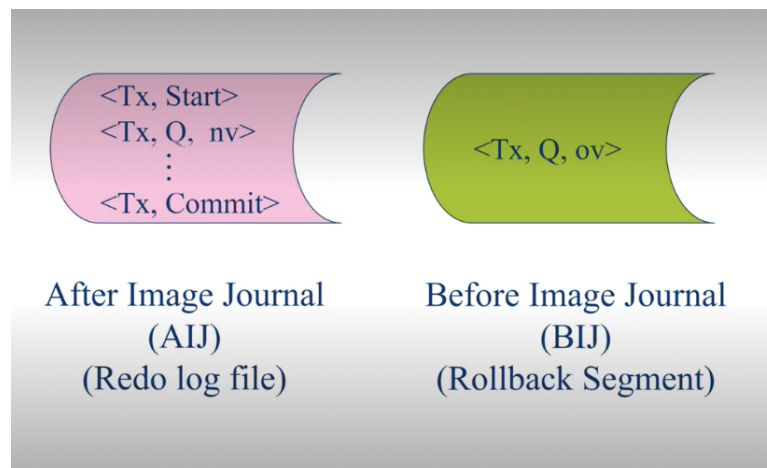
Immediate Database Modification Example		
Log	Write	Output
<T ₀ start>		
<T ₀ , A, 1000, 950>		
<T ₀ , B, 2000, 2050>		
	A = 950 B = 2050	
<T ₀ commit>		
<T ₁ start>		
<T ₁ , C, 700, 600>		
	C = 600	
<T ₁ commit>		
		B _B , B _C B _A
		B _C output before T ₁ commits B _A output after T ₀ commits

■ Note: B_X denotes block containing X.

Output may take place before or after commit.



in here, $\langle Tx, O, ov, nv \rangle$ is in the same log file, but actually it possible and quit popular that we separate old and new value



- After Image Journal - separate the modify one
- Before Image Journal - the old value

most implementation nowadays → separated.

Originally intention on why we separated them → space

- old value can be recycle after commit → save the space
- we can recycle the old value after each commit

point

- space → old value can be recycle after commit
- we have the option to install 1 of them or both
 - for each, logically, you can still have redo and roll back

Recovery Process

- When you run transaction, log record go to log buffer first, then log file.
- Concurrently, DB buffer move between buffer and DB space (input and output)

And suppose you have a system crash

- what should we do?

Log-based Recovery requires the write ahead protocol or write ahead logging (WAL)

- which said that
 - WAL → log record must be output to the log file (stable storage) before modified buffer can be output to the db space.
 - otherwise, you can't recover, this is a big requirement.
 - And it very clear that this is for DBMS, DBMS must follow WAL.
 - but remembered, the DBMS must tell the OS to do so, the DBMS it self does not work directly with the disk. DBMS informed the OS, it test together for WAL.

** System crashed

- Our response →
 - fix the physical problem.
 - restart OS, restart DBMS
 - when we restart DBMS, the DBMS recovery manager (a module of the DBMS) performs the following task
 1. read the log file from the end to determine which transaction commit, which one failed.
 - transaction that commit you will find commit record, if you can't find it → transaction failed
 2. For those failed Txns, Txids are kept in the undo list.
For those committed Txns, Txids are kept in the redo list.

- After checking, we know which one commit, which one failed. (we check the commit record)
3. For each transaction in the undo list, use the ovs to undo the DB space (solve the second problem, output before commit)
- for those transaction that failed → we undo all of them. ឧបាយដោះស្រាយ 7 លែង system failed ៣០ 4 លែង failed 20 transactions → undo ក្នុង

For each transaction in the redo list, use the nvs to redo the DB space (solve the first problem) (commit before output)

- for the redo, how far back should we redo the DB space.
 - assume 100,000 transactions already commit, but we don't know which one already output → do we need to redo all of them.
 - in principle, redo only those who not yet output.
 - the challenging part → we don't know which one already output (ខ្ញុំដឹងតែ commit ហើយ)
 - you can said that we need a check points
 - check point are required to determine Tx's that have already output changes to the DB space. (we need to refer to some check point)